



Practica 6 - Ejercicios sobre pilas y colas.

Algoritmos y Programación (Universidad Nacional Arturo Jauretche)

Algoritmos y Programación

Práctica 6

- 1) Implemente la clase Pila con la siguiente especificación:

push(elemento)	Agrega un elemento a la pila.
pop()	Extraer un elemento de pila.
isEmpty()	Retorna true si pila está vacía.
top()	Retorna el elemento al tope de la pila sin sacarlo.
size()	Retorna la cantidad de elementos que tiene pila.
reverse()	Retorna una nueva pila con los elementos en posición invertida (el primero en último lugar).
pushAll(otrapila)	Agrega a la pila receptora todos los elementos de otrapila (respetando el orden original de salida).

- 2) Implemente la clase Cola con la siguiente especificación:

push(elemento)	Agrega un elemento a la cola.
pop()	Extraer un elemento de cola.
isEmpty()	Retorna true si cola está vacía.
top()	Retorna el elemento al tope de la cola sin sacarlo.
size()	Retorna la cantidad de elementos que tiene cola.
reverse()	Retorna una nueva cola con los elementos en posición invertida (el primero en último lugar).
pushAll(otracola)	Agrega a la cola receptora todos los elementos de otracola (respetando el orden original de salida).

- 3) Escriba una función que reciba una string y devuelve el string invertido utilizando una Pila.

- 4) Escriba un función que reciba un string y devuelva true si la cadena esta balanceada o false en caso contrario.

Una cadena de caracteres S está balanceada si tiene alguna de las siguientes formas:

S = "" (string de longitud cero)

S = "(T)"

S = "[T]"

S = "{T}"

S = "TU"

Donde T y U son cadenas balanceadas.

Por ejemplo, "{ [() () } { [] }" está balanceada, pero " ([])" no lo está.

¿Qué colección de datos (Pila o Cola) utilizaría para resolver este problema?

- 5) Evaluación de expresiones postfijas :

Una expresión aritmética en notación postfija (o notación polaca inversa) es una secuencia formada por símbolos de dos tipos diferentes: operadores (para simplificar, consideraremos únicamente los operadores aritméticos binarios +, -, * y /) y operandos (valores entre 0 y 9). Cada operador se escribe detrás de sus operandos.

Por ejemplo, a la expresión siguiente, escrita en la notación habitual (infija):

$$4 * 6 / 3$$

le corresponde la siguiente expresión en notación postfija:

$$4\ 6\ *\ 3\ /\$$

Escriba una función que reciba un string que representa una expresión en notación postfija devuelva el resultado de su evaluación. En el ejemplo anterior el resultado evaluar la expresión sería 8.

¿Qué colección de datos (Pila o Cola) utilizaría para resolver este problema?

- 6) Utilizando la clase Cola, escriba una función llamada que reciba 2 Colas con números enteros y devuelva una nueva Cola con sus elementos sumados uno a uno.

Ej:

Cola A	Cola B	Cola Resultado
3	6	9
4	2	6
2	9	11
8	11	19
12	3	15

- 7) Implemente una aplicación que permita manejar colas de servicios para una compañía de seguros. Un cliente llega y según lo que necesite le corresponde un número dentro de una de las colas. El personal de atención va llamando a cada cliente eligiendo la cola que desea atender.

Este sistema deberá permitir simular la llegada de un cliente ingresando por consola la letra C seguida de un número de servicio. El sistema deberá entregar un número de atención (el que corresponda según la cola actual del servicio).

También deberá permitir atender a un cliente ingresando por consola la letra A seguida del número de servicio. El sistema deberá mostrar en consola el número que se está llamando.

Utilice la clase Cola para la resolución de este sistema.

- 8) Implemente utilizando la clase Pila para la resolución del juego de las Torres de Hanoi para tres discos.
- 9) Intente generalizar el juego de las Torres de Hanoi para n discos.
- 10) Implemente un TAD Conjunto con la siguiente especificación:

agregar(elemento)	Agrega un elemento al conjunto.
pertenece(elemento)	Retorna true si el elemento está en el conjunto.
esVacio()	Retorna true si el conjunto está vacío.
cardinalidad()	Devuelve la cantidad de elementos del conjunto
union(conj)	Retorna un nuevo conjunto con la unión del conjunto receptor y conj.
interseccion(conj)	Retorna un nuevo conjunto con la interseccion del conjunto receptor y conj.
diferencia(conj)	Retorna un nuevo conjunto con la diferencia entre el conjunto receptor y conj.
incluye(conj)	Devuelve true si el conjunto receptor incluye a conj.
incluido(conj)	Devuelve true si el conjunto receptor está incluido en conj.

Además redefina los métodos:

<code>ToString()</code>	Devuelve un string con el siguiente formato: <code>{elem1, elem2, elem3, elem4}</code>
<code>Equals(conj)</code>	Usado para la comparación entre conjuntos, dos conjuntos son iguales si poseen los mismos elementos.